Multi-Robot Planning with Model Predictive Control

Based on: "A Benchmark for Multi-Robot Planning in Realistic, Complex and Cluttered Environments"

Seminar Electromobility WS 2024/25

Sara Bavi
M.Sc. Automation & Control
Email: gux89nar@rptu.de
Institute of Electro mobility

Ammar Said
M.Sc. Commercial Vehicle Technology
Email: feq66fem@rptu.de
Institute of Electro mobility

Introduction

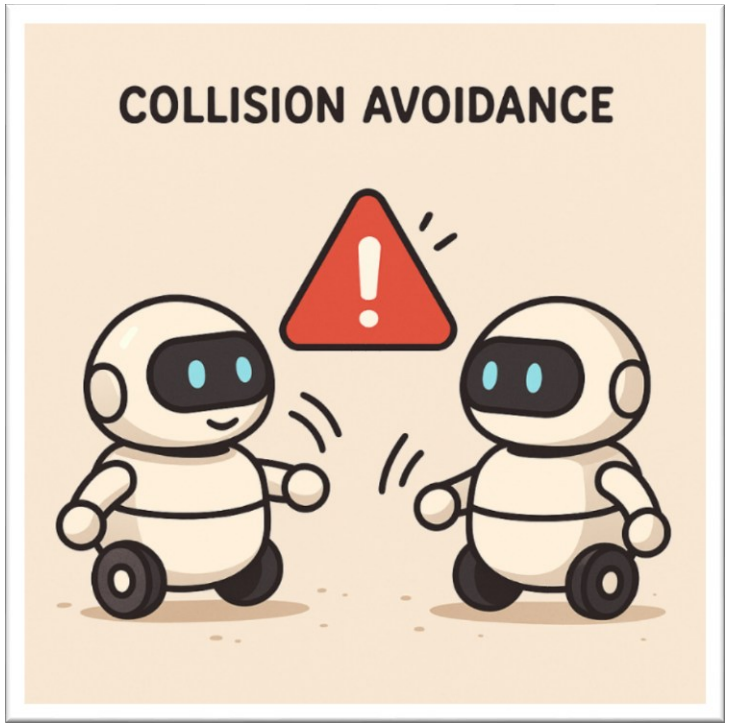
- **Model Predictive Control (MPC)** is an advanced control strategy that optimizes control actions by solving a finite-horizon optimization problem at each time step. It is widely used in robotics for real-time decision-making, motion planning, and dynamic control.

- **Benchmark for Multi-Robot Planning in Realistic, Complex and Cluttered Environments**
  This paper introduces an open-source benchmark suite designed to evaluate and compare multi-robot planning and coordination systems in realistic 3D simulated environments, such as warehouses and homes. The framework helps users test and compare different robot planning methods, making it easier to choose the best one for real-life use

Introduction

Objective of this Presentation:

- Summarize key insights from the paper *"A Benchmark for Multi-Robot Planning in Realistic, Complex, and Cluttered Environments."*
- Explore how MPC can enhance multi-robot path planning by addressing challenges such as real-time execution uncertainty and dynamic obstacles.
- Introduce the fundamentals of Model Predictive Control (MPC) and its relevance in multi-robot planning.



Overview of a Multi-Robot Planning Benchmark

Multi-Robot Planning and Coordination: plays a vital role in real-world applications such as warehouse automation, autonomous vehicles, and search-and-rescue operations. Effective path planning ensures safe, collision-free, and synchronized navigation of multiple robots within dynamic and cluttered environments.

MRP-Benchmark: An open-source benchmarking framework for evaluating multi-robot planning algorithms in ROS 2 and Gazebo, Nav2 providing realistic 3D simulations for algorithm comparison.

Overview of a Multi-Robot Planning Benchmark

What is Multi-Agent Path Finding (MAPF)?

- A fundamental problem in multi-robot coordination.
- Focuses on finding collision-free paths for multiple robots.
- Crucial for warehouses, autonomous vehicles, and industrial automation.

Challenges in MAPF:

- Navigating in congested and dynamic environments.
- Ensuring optimal task allocation and efficient motion planning.
- Balancing computation time vs. path efficiency.

Task Assignment in Multi-Robot Planning Benchmark

Benchmarking Multi-Robot Path Planning: MRP-Bench evaluates both task assignment and path finding, two tightly coupled components of multi-robot planning.

Task Assignment:

Each robot is assigned to a goal by minimizing the total cost using a cost matrix

Cost can be calculated using:

- Euclidean distance
- Manhattan distance
- A*-based path length

Represented as a bipartite graph:

- Robots and goals are nodes.
- Edges weighted by cost.

Matching problem solved via optimization to minimize total path cost.

Path Finding in Multi-Robot Planning Benchmark

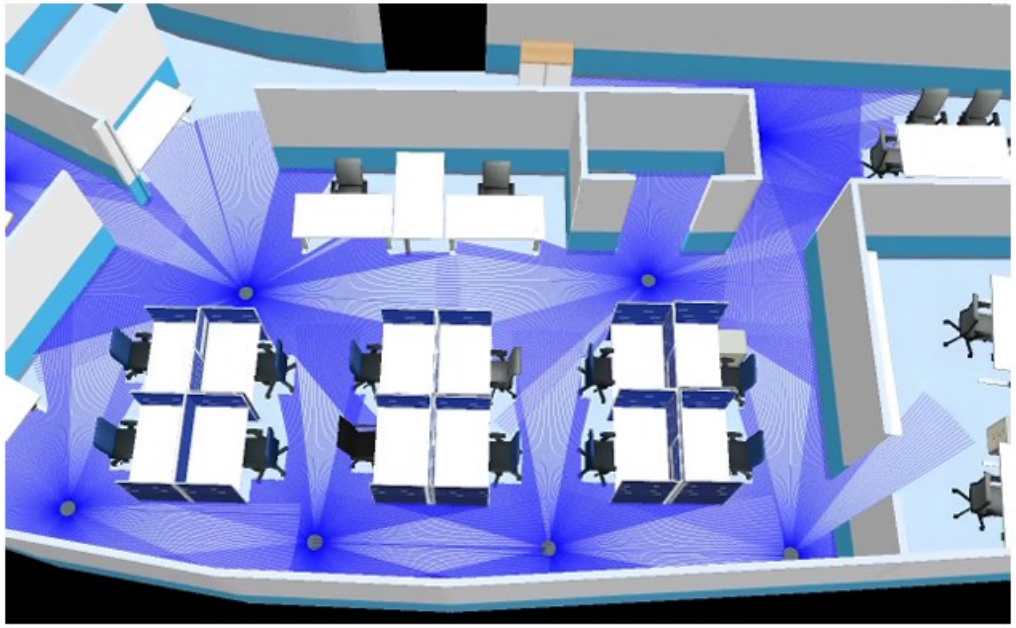
Path Finding:

Guide multiple robots from their start positions to goal locations without collisions.

- **Environment Modeling:**
  The environment is represented as a grid or an undirected graph.
  Robots can move to adjacent cells or wait in place.
- **Robots must avoid:**
  - Being in the same cell at the same time (vertex conflict)
  - Swapping positions at the same time (edge conflict)
- **Path Planning:**
  Algorithms generate individual paths (schedules) for each robot, ensuring all constraints are met.
- **Evaluation Metrics:**
  - Total cost: Sum of all robot path lengths
  - Makespan: Time taken for the last robot to reach its goal

Benchmark Initializing

- Nine robots navigate an office environment (some positioned behind walls)
- Laser scanner rays from each robot are visualized in blue
- Compatible with Gazebo, ROS 2, and Nav2 — supports all robot models within these platforms
- Initial experiments conducted with TurtleBot3
- Small-sized robot
- Equipped with a simple 2D LiDAR
- Cost-effective and widely accessible



Evaluation Metrics in Benchmark

In benchmark use two common sets of metrics to evaluate the performance:

Planning Metrics:

- Success Rate
- Planning Time
- Time to complete the task
- Cost

Execution Metrics:

Analyzed from recorded data during scenario execution:

- Execution Time: Time taken for all robots to reach their goals. A timeout can be set.
- Goals Reached: Number of robots that reached their goal within the timeout.
- Minimum Distance Between Robots: Tracks how close robots get to each other without colliding.
- Blocked Time per Robot: Measures how long a robot is stuck without moving, using a sliding window approach.

Comparison of Algorithms: Replicability Experiment (100 Runs)

To evaluate the performance consistency and replicability of different algorithms across 100 runs in various scenarios.

- **Algorithms Tested:**

A*

CBS

ECBS

EECBS

- **Experiment Setup:**

Run 100 times for each algorithm and scenario.

Environment: Warehouse map with 5 agents and a 300s time limit for completion.

Goal: Test consistency and success rate in producing valid schedules.

Comparison & Performance Insights

Algorithm	Strengths	Weaknesses
A*	Simple, Fast	No collision handling
CBS	Optimal	Computationally expensive
ECBS	Faster than CBS	Less optimal
EECBS	More efficient	Trade-off between speed and optimality

Key Results:

CBS is accurate but slow under high complexity.

ECBS and EECBS perform better overall in practical settings.

Future Improvements

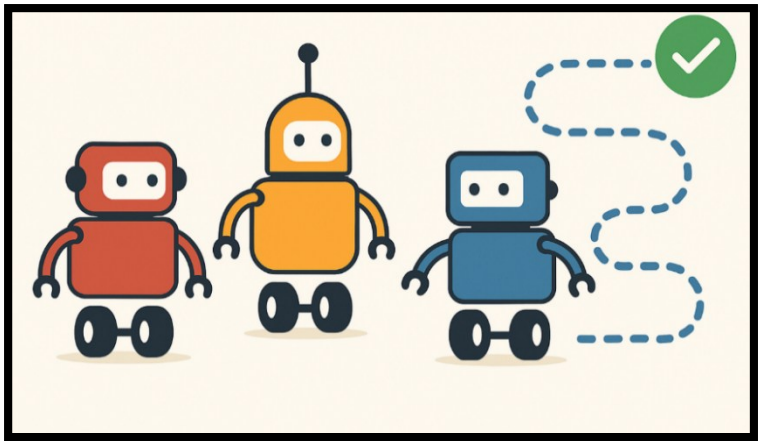
Support more robots in real-time.

Introduce dynamic obstacles.

How MPC Can Improve Multi-Robot Path Planning

Challenges in Current Path Planning Methods:

- **Dynamic Obstacles:** Current algorithms (A*, CBS, ECBS) fail to address real-time changes in the environment.
- **Execution Uncertainty:** Path planning algorithms do not handle disturbances such as control errors or unforeseen obstacles.
- **Lack of Real-Time Adaptation:** Predefined, static environments are assumed during planning, which is not realistic.



MPC's Role in Enhancing the Approach

- **Bridging the Gap Between Planning & Execution:**
  - Ensures smooth, optimal trajectory execution in real-time.
  - Handles dynamic obstacles and unexpected disturbances.
- **Efficiently Managing Constraints:**
  - Incorporates velocity, angular, and other kinematic constraints into optimization.
  - Ensures safe, feasible control actions throughout the path.

Layout

Main scopes of talk:

1. Model predictive control (MPC)
2. Control loop (Architecture)
3. State-space representation (kinematics model)
4. Control objectives
5. Numerical implementation (MPC formulation)
6. Collision avoidance
7. Conclusion and future work

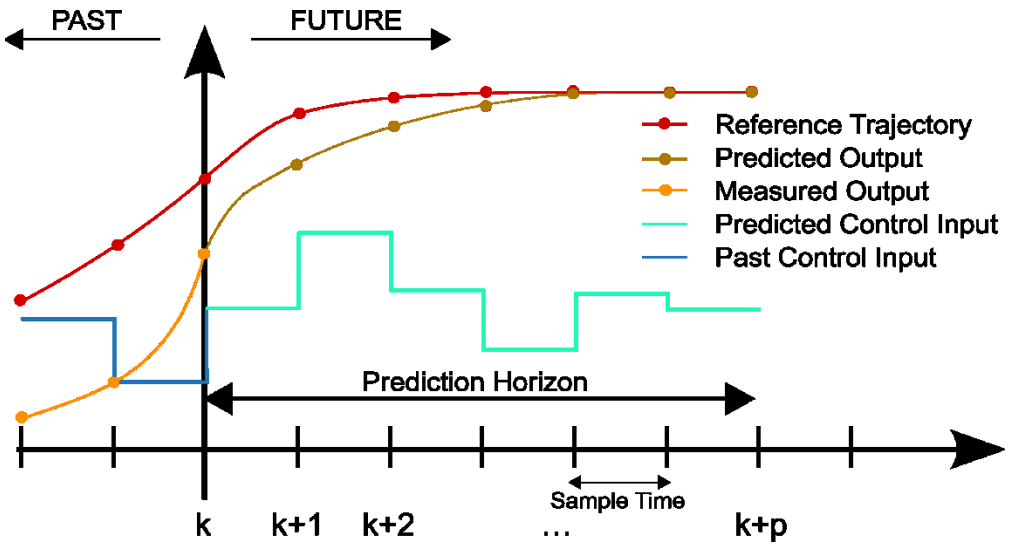
Model Predictive Control (MPC)

About MPC

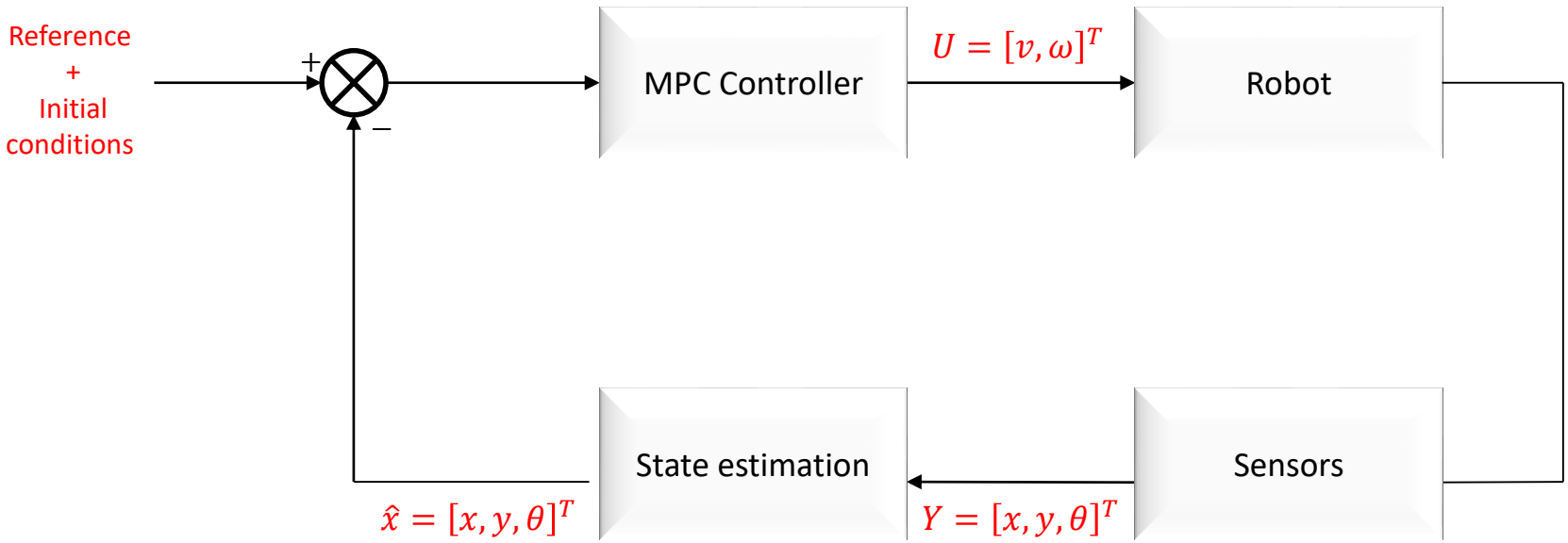
- Generally applicable to MIMO systems
- Natural consideration of both states and control constraints
- Approximately optimal control (System Stabilization!)
- Requires online optimization

Main Parameters

- Sampling Time (Δt)
- Prediction Length (N)
- Input/Control Variables (U)
- State Variables (X)
- Output Variables (Y)
- Weighting Matrices (Q, R)



Control Loop (Architecture)



State-Space Representation

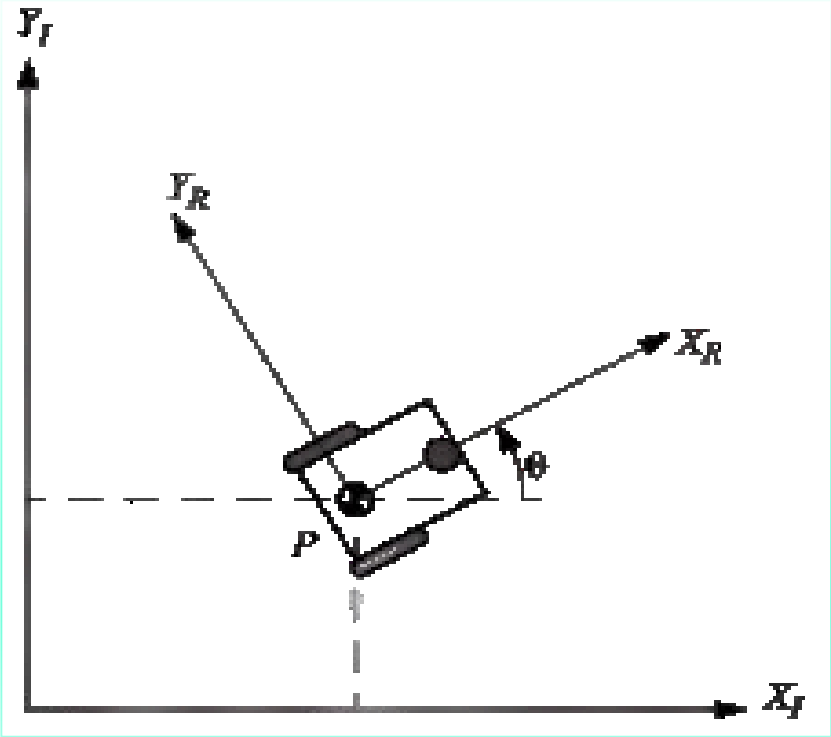
❖ Non-Holonomic differential Mobile Robot (3 D.O.F.)

$$\frac{dx}{dt} = f(x(t), u(t))$$

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \frac{r}{2} \begin{bmatrix} (\emptyset^r + \emptyset^i)\cos(\theta) \\ (\emptyset^r + \emptyset^i)\sin(\theta) \\ (\emptyset^r - \emptyset^i) / D \end{bmatrix}$$

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} \frac{r}{2} & (\emptyset^r + \emptyset^i) \\ \frac{r}{2D} & (\emptyset^r - \emptyset^i) \end{bmatrix}$$

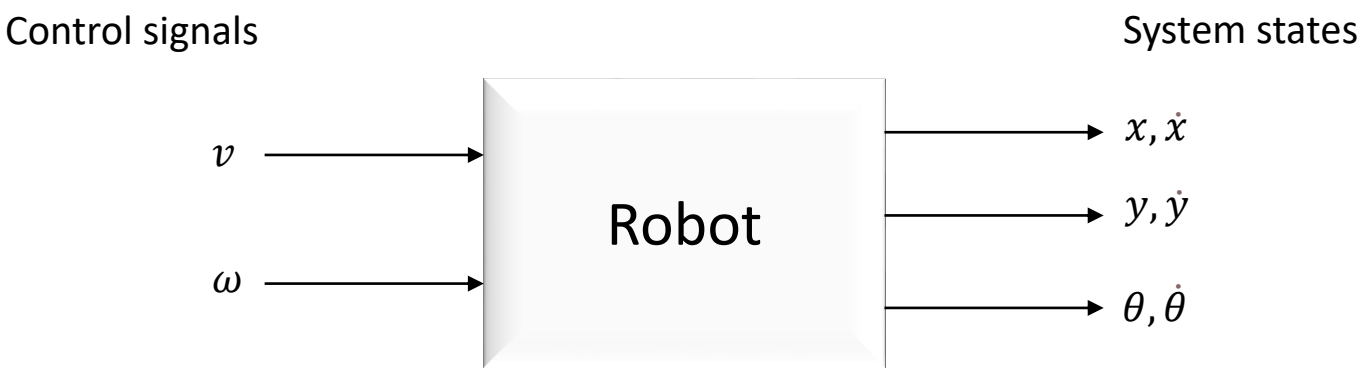
$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$



State-Space Representation

❖ **View from input/output perspective**

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

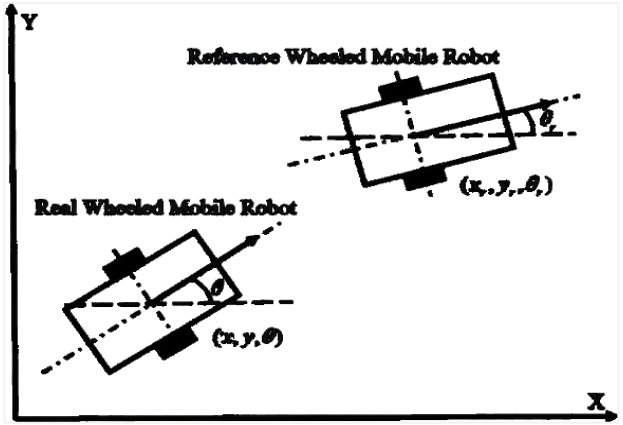


Control Objectives

❖ Point Stabilization

Constant reference states over the prediction horizon.

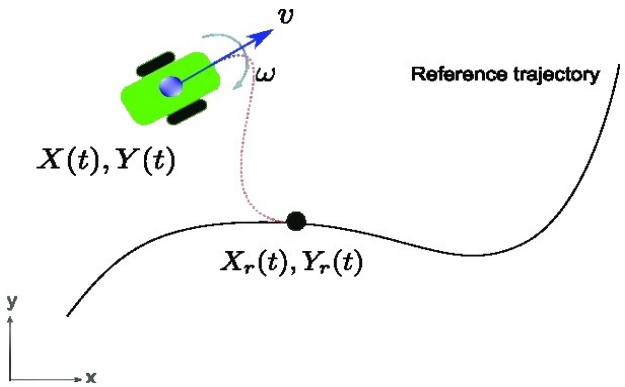
$$\begin{bmatrix} x_r(t) \\ y_r(t) \\ \theta_r(t) \end{bmatrix} = \begin{bmatrix} x_d \\ y_d \\ \theta_d \end{bmatrix}, \forall t$$



❖ Trajectory Tracking

Varying reference states over the prediction horizon.

$$\begin{bmatrix} x_r(t) \\ y_r(t) \\ \theta_r(t) \end{bmatrix} = \begin{bmatrix} x_d(t) \\ y_d(t) \\ \theta_d(t) \end{bmatrix}, \forall t$$



Optimal Control Problem

$$L(x,u) = \|X_u - X_r\|_Q^2 + \|u\|_R^2$$

Running Cost

$$\min J_n(x_o,u) = \sum_{k=o}^{N-1} L\big(x_u(k),u(k)\big)$$

Cost Function

Subject to:

$$x_u(k+1) = f\big(x_k(k),u(k)\big)$$

Kinematics Model

$$x_u(0) = x_0$$

Initial State

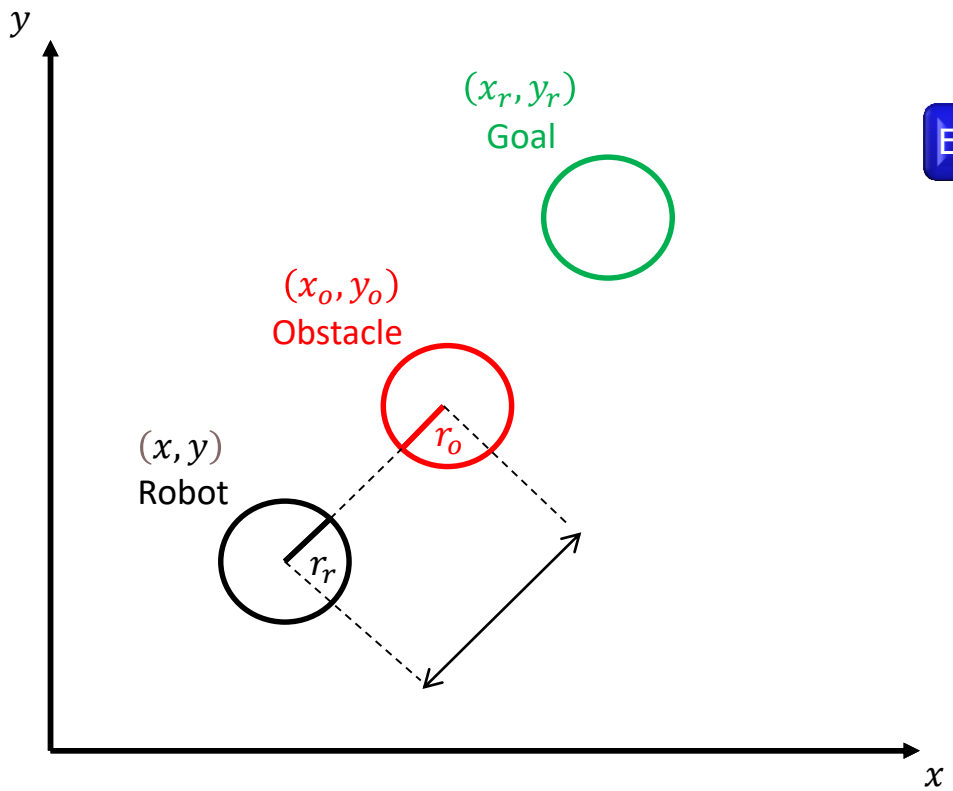
$$u(K) \in U, \forall k \in [0, N-1]$$

Control Boundaries

$$x_u(k) \in X, \forall k \in [0, N]$$

State Boundaries

Collision Avoidance



Euclidean distance:

$$\sqrt{(x - x_o)^2 + (y - y_o)^2} \geq r_r + r_0$$

Remarks:

- Assuming static obstacle, then (x_o, y_o) is constant all over the prediction horizon (N).
- The predicted states (x, y) should respect the irregularity path constraint all over the prediction horizon (N).

Conclusion and Future Work

Conclusion

- ✓ Path planning + Control

  Path planning alone is insufficient; robots need real-time control mechanisms.

- ✓ Real-Time Adaptability

  MPC ensures collision-free navigation in dynamic environments.

Future Work

- When does MPC stabilize the system?
- How good is the performance of the MPC feedback law?
- How long does the optimization horizon (N) need to be?

Thank you for listening!

Questions?